

Przedmiot	Mikroprocesory - PROJEKT	
Prowadzący		
Temat	<i>Projekt gry na stm32</i>	
Student (+ nr indeksu)		
Email		
Data wykonania	12.02.2025 r.	
Data oddania	12.02.2025 r.	

Specyfikacja projektu:

1. Oprogramować komunikację mikroprocesora z PC poprzez interfejs asynchroniczny z wykorzystaniem przerwań i buforów kołowych.
2. Zaprojektować i zaimplementować protokół komunikacyjny pozwalający na adresowanie ramek, przekazywanie dowolnych danych, weryfikację poprawności przesłanych danych z uwzględnieniem ich kolejności.
3.
 - a) Obsłużyć joystick analogowy poprzez przetwornik adc, dwa kanały, częstotliwość próbkowania 1 kilo Hertz.
 - b) ADC wspierane przez DMA (Direct Memory Access).
 - c) Zaimplementować obsługę gry.

PROTOKÓŁ KOMUNIKACYJNY

NAZWA ->	Początek ramki [	Nadawca	Odbiorca	Komenda	Dane	Suma kontrolna CRC16 MODBUS	Koniec ramki]
Sposób przetwarzania	char	char	char	char	char	char	char
Ilość bajtów	1 bajt	2 bajty	2 bajty	0-10 bajtów	0-200 bajtów Wszystkie znaki	4 bajty znaki HEX	1 bajt

W protokole dane są przesyłane jako ciągi bajtów. W standardzie ASCII każdy znak zajmuje jeden bajt.

Wartości liczbowe wykorzystywane jako dane należy przysyłać w postaci decymalnej przy pomocy znaków (ASCII).

Początek ramki – detekcja ramki zaczyna się od znaku “[” i tylko po wykryciu tego znaku zacznie się odbiór

Nadawca – tutaj mogą występować wszystkie znaki oraz nadawca może być dowolny

Odbiorca – również mogą występować wszystkie znaki, zdefiniowany jest jako “MC”

Komenda – występują wszystkie znaki, przykładowe komendy podane są w dalszej części protokołu i zajmują od 0 do 10 bajtów

Dane - występują wszystkie znaki, zajmują od 0-200 bajtów

Suma kontrolna – zapisywana na 4 bajtach jako wartość HEX

Koniec ramki - odbiór ramki kończy się po napotkaniu znaku “]”

Znaki specjalne i ich zamiana

Niektóre znaki są traktowane jako zabronione i muszą być odpowiednio zakodowane, aby mogły być przesyłane w ramce protokołu. Zabronione znaki nie mogą występować w danych samodzielnie. Pełna ramka może mieć maksymalnie 220 znaków, a minimalna (bez danych i komendy) 10 znaków.

Lista znaków zabronionych i ich kodowanie:

[- zastępowane jest sekwencją \1

] - zastępowane jest sekwencją \2

\ - zastępowane jest sekwencją \3

Suma kontrolna:

Liczona jest dla całej zawartości ramki z wyłączeniem znaku startu i znaku stopu, czyli dla nadawcy, odbiorcy, komendy, danych. Liczona jest za pomocą CRC 16-MODBUS. Inicjalizacja jest od 0xFFFF – czyli wartość początkowo używana w algorytmie modbus. Dla każdego bajtu algorytm przetwarza 8 bitów. Sprawdzany jest LSB (least significant bit) czyli bit najmłodszy ustawiony na samym końcu. Jeśli bit ten wynosi 1 (jest ustawiony), wartość CRC jest przesuwana w prawo o 1 bit, a następnie wykonywana jest operacja XOR z wielomianem 0xA001 (Wielomian 0xA001 to odwrócona wersja wielomianu 0x8005, stosowany w algorytmie Modbus). Jeśli bit ten wynosi 0, wartość CRC jest jedynie przesuwana w prawo o 1 bit. CRC jest przedstawione jako 4 znakowa sekwencja szesnastkowa.

KOMENDY:

<i>komenda</i>	<i>opis</i>
STOP	Stopuje gre
RESUME	Wznawia gre
BTNSTATE	Zwraca aktualny stan przycisku joysticka (0 – niewciśnięty, 1 – wciśnięty)
POS	Zwraca aktualne wychylenie joysticka

STOP (Stopowanie gry) -

Ilość znaków: 1 (początek ramki) + 2 (Nadawca) + 2 (Odbiorca) + 4 (STOP) + 4 (CRC) + 1 (Koniec ramki)

Przykład ramki: [PCMCSTOP7919]

Ilość znaków jako wynik: 1 (początek ramki) + 2 (Nadawca) + 2 (Odbiorca) + 4 (STOP) + 6 (PAUSED) + 4 (CRC) + 1 (Koniec ramki)

RESUME (Wznowienie gry) -

Ilość znaków: 1 (początek ramki) + 2 (Nadawca) + 2 (Odbiorca) + 6 (RESUME) + 4 (CRC) + 1 (Koniec ramki)

Przykład ramki: [PCMCRESUME92D4]

Ilość znaków jako wynik: 1 (początek ramki) + 2 (Nadawca) + 2 (Odbiorca) + 6 (RESUME) + 7 (STARTED) + 4 (CRC) + 1 (Koniec ramki)

BTNSTATE (Zwrot stanu przycisku) -

Ilość znaków: 1 (początek ramki) + 2 (Nadawca) + 2 (Odbiorca) + 8 (BTNSTATE) + 4 (CRC) + 1 (Koniec ramki)

Przykład ramki: [PCMCBTNSTATE83EC]

Ilość znaków jako wynik: 1 (początek ramki) + 2 (Nadawca) + 2 (Odbiorca) + 8 (BTNSTATE) + 4 (BTN i liczba 1 lub 0) + 4 (CRC) + 1 (Koniec ramki)

POS (Zwraca współrzędne X i Y) -

Ilość znaków: 1 (początek ramki) + 2 (Nadawca) + 2 (Odbiorca) + 3 (POS) + 4 (CRC) + 1 (Koniec ramki)

Przykład ramki: [PCMCPOS6008]

Ilość znaków jako wynik: 1 (początek ramki) + 2 (Nadawca) + 2 (Odbiorca) + 3 (POS) + 10(X2048Y4095) + 4 (CRC) + 1 (Koniec ramki)

BŁĘDY:

<i>błąd</i>	<i>opis</i>
INVALID_SENDER_RECIVER	Zły nadawca lub odbiorca
INVALID_COMMAND	Nierozpoznana komenda
INVALID_CRC	Błędne CRC

ODBIÓR DANYCH

Odbieranie danych przez UART realizuje na przerwaniach NVIC oraz zapisywaniu ich do buforu kołowego.

OBSŁUGA PRZERWAŃ

Odbieranie znaków:

Na początek włączam przerwania (dla pierwszego znaku) metodą z biblioteki HAL zapisując ją w **main()** programu:

```
HAL_UART_Receive_IT(&huart2, &USART_RxBuf[USART_RX_Empty], 1);
```

Kod odpowiedzialny za obsługę przerwania:

```

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart == &huart2) {
        USART_RX_Empty++;
        if (USART_RX_Empty >= USART_RXBUF_LEN) USART_RX_Empty = 0;
        HAL_UART_Receive_IT(&huart2, &USART_RxBuf[USART_RX_Empty], 1);
    }
}

```

Funkcja ta zapisuje znak do bufora kołowego i wykonuje kolejne przerwania, dopóki przychodzą znaki do zapisu.

Wysyłanie znaków:

Analogicznie wysyłam znaki, czyli tworzę przerwanie dla pierwszego, nieodczytanego znaku z bufora nadawczego przy pomocy metody z biblioteki HAL:

```

HAL_UART_Transmit_IT(&huart2, &tmp, 1);

```

W obsłudze tego przerwania wysyłam kolejne znaki, dopóki wskaźniki USART_TX_Empty i USART_TX_Busy nie będą równe, co oznacza, że w buforze nie ma już więcej danych do wysłania:

```

void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart == &huart2) {
        if (USART_TX_Empty != USART_TX_Busy) {
            uint8_t tmp = USART_TxBuf[USART_TX_Busy];
            USART_TX_Busy++;
            if (USART_TX_Busy >= USART_TXBUF_LEN) USART_TX_Busy = 0;
            HAL_UART_Transmit_IT(&huart2, &tmp, 1);
        }
    }
}

```

W celu zapisania danych do bufora nadawczego i zainicjalizowania transmisji w trybie asynchronicznym przez UART posługuje się funkcją USART_fsend

```

void USART_fsend(char* format, ...)
{
    char tmp_rs[128];
    int i;
    __IO int idx;
    va_list arglist;

    va_start(arglist, format);
    vsprintf(tmp_rs, format, arglist);
    va_end(arglist);

    idx = USART_TX_Empty;
    for (i = 0; i < (int)strlen(tmp_rs); i++) {

```

```

USART_TxBuf[idx] = (uint8_t)tmp_rs[i];
idx++;
if (idx >= USART_TXBUF_LEN) idx = 0;
}

__disable_irq();
if ((USART_TX_Empty == USART_TX_Busy) &&
    (__HAL_UART_GET_FLAG(&huart2, UART_FLAG_TXE) == SET)) {
    USART_TX_Empty = idx;
    uint8_t tmp = USART_TxBuf[USART_TX_Busy];
    USART_TX_Busy++;
    if (USART_TX_Busy >= USART_TXBUF_LEN) USART_TX_Busy = 0;
    HAL_UART_Transmit_IT(&huart2, &tmp, 1);
} else {
    USART_TX_Empty = idx;
}
__enable_irq();
}

```

Ważnym jest, by przed wysłaniem pierwszego przerwania włączyć maskowanie przerwań przy pomocy **__disable_irq()**, aby zapobiec przypadkowej modyfikacji wskaźnika USART_TX_Busy.

Po zakończeniu transmisji od razu wyłączamy maskowanie przerwań, aby umożliwić obsługę kolejnych przerwań, takich jak np. odbiór danych.

ODBIÓR RAMEK

Funkcja getFrame wykorzystuje dwie inne funkcje jedną w celu sprawdzenia czy w buforze odbiorczym są jeszcze jakieś znaki (USART_kbhit) oraz do pobierania pojedynczy znaków z bufora odbiorczego (USART_getchar)

```

uint8_t* getFrame(size_t *outLength)
{
    static uint8_t frameBuf[FRAME_MAX_LENGTH];
    static uint16_t frameLen = 0;
    static FrameState frameState = WAITING_FOR_FRAME;

    while (USART_kbhit())
    {
        char c = (char)USART_getchar();
        if (c < 0)
        {

```

```
// Błąd/brak znaku
return NULL;
}

switch (frameState)
{
case WAITING_FOR_FRAME:
// Czekamy na '[' aby rozpocząć ramkę
if (c == '[')
{
frameState = COLLECTING_FRAME;
frameLen = 0;
frameBuf[frameLen++] = '[';
}
// Inne znaki poza ramką ignorujemy
break;

case COLLECTING_FRAME:
if (c == '\\')
{
// Używamy sekwencji escape
frameState = ESCAPING_CHARACTER;
}
else if (c == '[')
{
// Nowy '[' w trakcie ramki => reset i zaczynamy od nowa
frameLen = 0;
frameBuf[frameLen++] = '[';
}
else if (c == ']')
{
// Koniec ramki
if (frameLen < FRAME_MAX_LENGTH - 1)
{
frameBuf[frameLen++] = ']';
frameBuf[frameLen] = '\0'; // Terminator
*outLength = frameLen;

// Reset stanu do następnej ramki
frameState = WAITING_FOR_FRAME;
return frameBuf; // Zwracamy gotową ramkę
}
else
{
// Przepełnienie -> porzucamy ramkę
frameState = WAITING_FOR_FRAME;
frameLen = 0;
}
}
else
```

```

{
// Zwykły znak wewnątrz ramki
if (frameLen < FRAME_MAX_LENGTH - 1)
{
frameBuf[frameLen++] = (uint8_t)c;
}
else
{
// Przepełnienie
frameState = WAITING_FOR_FRAME;
frameLen = 0;
}
}
break;

case ESCAPING_CHARACTER:
{
// Interpretujemy znak po '\'
char decoded;
switch (c)
{
case '1': decoded = '['; break;
case '2': decoded = ']'; break;
case '3': decoded = '\\'; break;
default:
// Nieznana sekwencja -> porzucamy ramkę
frameState = WAITING_FOR_FRAME;
frameLen = 0;
continue;
}

// Wracamy do COLLECTING_FRAME
frameState = COLLECTING_FRAME;

// Dodajemy zdekodowany znak do bufora
if (frameLen < FRAME_MAX_LENGTH - 1)
{
frameBuf[frameLen++] = (uint8_t)decoded;
}
else
{
// Przepełnienie
frameState = WAITING_FOR_FRAME;
frameLen = 0;
}
break;
}

default:
// Bezpieczny reset
frameState = WAITING_FOR_FRAME;

```



```

frameLen = 0;
break;
}
}

// Brak pełnej ramki - NULL
return NULL;
}

```

Błędny początek: Ignoruje znaki przed '['. Ramka nie zaczyna się od innych znaków.

Restart na każdy '[': Jeśli w środku ramki pojawi się kolejny '[', kod **resetuje** ramkę (frameLen=0) i zaczyna nową od tego miejsca.

Znak escape: Jeśli trafi się '\\', wchodzimy w stan ESCAPING_CHARACTER.

- Jeśli kolejnym znakiem jest '1', '2', '3', konwertujemy go do znaku '[', ']', '\\'.
- Jeśli jest inny, uznajemy za błąd i porzucamy ramkę.

Zakończenie: ']' domyka ramkę – dopisujemy go do bufora, wstawiamy '\\0' i zwracamy.

Przepelnienie: Jeśli bufor osiągnie maksymalny rozmiar, **przerywamy** zbieranie i wracamy do oczekiwania.

Inne sekwencje (np. '\\X') – jeśli 'X' nie jest '1', '2', '3', **porzucamy** ramkę, by uniknąć bałaganu

PRZETWARZANIE RAMKI

```

void handleFrame(const uint8_t *frame, size_t length)
{
    if (length < 10)
    {
        return;
    }

    char asciiCrc[5];
    memcpy(asciiCrc, &frame[length - 5], 4);
    asciiCrc[4] = '\\0';

    int coreLen = (int)length - 6;
    if (coreLen < 0)
    {
        return;
    }

    char coreBuf[256];
    memcpy(coreBuf, &frame[1], coreLen);
    coreBuf[coreLen] = '\\0'; // NUL-terminacja
}

```

```
char sender[3];
memcpy(sender, coreBuf, 2);
sender[2] = '\0';

char receiver[3];
memcpy(receiver, &coreBuf[2], 2);
receiver[2] = '\0';

if (memcmp(receiver, "MC", 2) != 0)
{
    sendErrorResponse("MC", sender, "INVALID_RECEIVER");
    return;
}

const char *pCmdData = &coreBuf[4];

char command[11];
int i = 0;
while ((pCmdData[i] >= 'A') && (pCmdData[i] <= 'Z') && (i < 10))
{
    command[i] = pCmdData[i];
    i++;
}
command[i] = '\0';

const char *data = "";

uint16_t calcCrc = computeFrameCRC(sender, receiver, command, data);

char calcCrcStr[5];
uint16_to_ascii_hex(calcCrc, calcCrcStr);

if (memcmp(asciiCrc, calcCrcStr, 4) != 0)
{
    sendErrorResponse("MC", sender, "INVALID_CRC");
    return;
}

char outData[100];
processCommand(command, outData);
```

```
sendResponse(receiver, sender, command, outData);
}
```

Sekcja opisuje sposób analizy i obsługi ramki komunikacyjnej po jej odebraniu. Funkcja `handleFrame` sprawdza poprawność ramki pod kątem jej długości, struktury oraz zawartości pól (nadawca, odbiorca, komenda). Dodatkowo weryfikuje integralność danych poprzez porównanie obliczonej sumy kontrolnej CRC z wartością zawartą w ramce. W przypadku błędów, takich jak niewłaściwy odbiorca, komenda lub błędne CRC, generowana jest odpowiedź błędu.

Korzystam z `memcpy` oraz `memcmp` zamiast `strcpy`, `strcmp` ponieważ pozwalają na odczyt dowolnych danych i nie są ograniczone do momentu napotkania null-terminatora, w skrócie `memcpy` kopiuje blok pamięci o zadanej długości a `memcmp` porównuje blok pamięci o zadanej długości.

LICZENIE SUMY KONTROLNEJ

```
uint16_t calculate_crc(const uint8_t *data, size_t length)
{
    uint16_t crc = CRC_INIT;
    for (size_t i = 0; i < length; i++) {
        crc ^= (data[i] & 0x00FF);
        for (int bit = 0; bit < 8; bit++) {
            if (crc & 0x0001) {
                crc = (crc >> 1) ^ 0xA001;
            } else {
                crc >>= 1;
            }
        }
    }
    return crc;
}
```

Funkcja `calculate_crc` implementuje algorytm CRC-16 w standardzie Modbus do obliczania sumy kontrolnej ramki. Operacja obejmuje wszystkie pola ramki z wyłączeniem znaków początku ([]) i końca (]). Wynik jest prezentowany jako czteroznakowa sekwencja w formacie szesnastkowym. Działanie opisane jest wyżej w dokumentacji

PRZETWARZANIE KOMEND

```
void processCommand(const char *command, char *data)
{
    memset(data, 0, 100);

    if ( (memcmp(command, "STOP", 4) == 0) && (command[4] == '\0') )
    {
        gameTimerState = -1;
        sprintf(data, "PAUSED");
    }
}
```

```

}

else if ( (memcmp(command, "RESUME", 6) == 0) && (command[6] == '\0') )
{
gameTimerState = 1;
sprintf(data, "STARTED");
}

else if ( (memcmp(command, "BTNSTATE", 8) == 0) && (command[8] == '\0') )
{
GPIO_PinState pinState = HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_10);

if (pinState == GPIO_PIN_RESET)
{
sprintf(data, "BTN1");
}
else
{
sprintf(data, "BTN0");
}
}

else if ( (memcmp(command, "POS", 3) == 0) && (command[3] == '\0') )
{
sprintf(data, "X%uY%u", (unsigned int)joystick[0], (unsigned int)joystick[1]);
}

else
{
sprintf(data, "INVALID_COMMAND");
}
}

```

Na samym początku zerujemy bufor data by usunąć ewentualne” śmieci” które mogłyby zostać w pamięci.

Korzystam z sprintf ponieważ pozwala on formatować, czyli łączyć wartości, wyrazy, itd. w jeden łańcuch, dzięki czemu np w komendzie POS mogę w łatwy sposób zwrócić wartości odczytane z osi joysticka.

Korzystam z unsigned int ponieważ %u formatuje liczbę bez znaku, więc rzutowanie unsigned zapewnia poprawne dopasowanie typu.

W MX_GPIO_Init() ustawiam GPIO_NOPULL na GPIO_PULLUP, dzięki temu w stanie puszczonego pin będzie widział 1 (SET) a przy wycisnięciu (zwieranie do masy) będzie 0 (RESET), komendy są napisane na odwrót żeby wcisnięty zwracał prawdę czyli BTN1

WYSYŁANIE RAMKI:

```

void sendResponse(const char *sender, const char *receiver,
const char *command, const char *data)
{

uint16_t crcVal = computeFrameCRC(sender, receiver, command, data);
char crcAscii[5];
uint16_to_ascii_hex(crcVal, crcAscii);


char frameCore[119];

int coreLen = snprintf(frameCore, sizeof(frameCore),
"%s%s%s%s", sender, receiver, command, data);

if (coreLen < 0 || coreLen > (FRAME_MAX_LENGTH - 6))
{

return;
}

char fullFrame[240];
int ffLen = snprintf(fullFrame, sizeof(fullFrame),
"%s%s]", frameCore, crcAscii);
if (ffLen < 0 || ffLen >= (int)sizeof(fullFrame))
{

return;
}

USART_fsend("%s", fullFrame);
}

```

```

abdcS/1/1//1//3/3/4/2/2//22[PCMCSTOP7919]

[MCPCSTOPPAUSED34D]

ABVSDYUDYUV[dasuihduas[PCMCRESUME92D4]

[MCPCRESUMESTARTEDF5FD]

[PCMCP0S6008]

[MCPCPOSX3966Y396734AA]

dshauias[PCMCP0S6008]

[MCPCPOSX3036Y05DF8]

```

```
[\1CMCSTOPCA58]
[MC[CSTOPPAUSED08A8]
[P\2MCRESUME78FC]
[MCP] INVALID_CRC A A 8 4]
[P\2MCRESUME3255]
[MCP] RESUMESTARTED E B E 3]
```

Funkcja sendResponse:

1. Oblicza CRC na podstawie nadawcy, odbiorcy, komendy i danych.
2. Składa "core" ramki i tworzy jej pełną wersję, dodając znaki otwierające/zamykające oraz CRC.
3. Sprawdza limity długości, aby zapobiec przepełnieniu bufora.
4. Wysyła ramkę przez UART w formacie zgodnym z protokołem komunikacyjnym.

DMA - Direct memory access

Sterowanie grą odbywa się przy pomocy przetwornika analogowo-cyfrowego (Thumb joystick). Informacja o pozycji gałki w danej chwili jest dostępna poprzez wysyłanie odpowiedniego sygnału analogowego z joysticka (joystick jest zbudowany z dwóch potencjometrów, po jednym na oś co pozwala na manipulację rezystancją prądową, a w efekcie wysyłanie różnego napięcia w zależności od położenia gałki joysticka) do przetwornika mikrokontrolera. W przetworniku sygnał przechodzi proces kwantyzacji i wartość napięcia jest zmieniana, na konkretną wartość z przedziału <0, 4095> (czujnik mikrokontrolera jest 12 bitowy). Wynik ten przy pomocy DMA w trybie cyklicznym możemy wysyłać do np. zmiennej programu.

Częstotliwość próbkowania 1 kilohertz

Do obliczenia prawidłowego próbkowania postępujemy się wzorem

$$\text{generowanie impulsów PWM} = \frac{8\text{MHz}}{[(0+1)*(7999+1)*(0+1)]} = 1 \text{ kilohertz}$$

8MHz – to częstotliwość bazowa timera, pochodząca z zegara mikrokontrolera

PSC - czyli prescaler który służy do podziału częstotliwość bazowej u mnie ustawiony jest na 0

ARR – Auto-Reload Register timer zlicza do wartości ustawionej co oznacza że pełny cykl trwania w moim przypadku 7999+1 kroków.

RCR – Repetition Counter czyli zdarzenie aktualizacji generowane jest po każdym cyklu licznika

Interpretacja wyniku – timer generuje zdarzenie aktualizacji co 1ms, co jest zgodne z obliczoną częstotliwością 1 kilohertz

Configuration

Reset Configuration

NVIC Settings

DMA Settings

GPIO Settings

Parameter Settings

User Constants

Configure the below parameters :

Search (Ctrl+F)

ADCs_Common_Settings

ModeIndependent mode

ADC_Settings

Data AlignmentRight alignment

Scan Conversion ModeEnabled

Continuous Conversion M...Disabled

Discontinuous Conversio...Disabled

ADC_Regular_ConversionMode

Enable Regular Conversio...Enable

Number Of Conversion2

External Trigger Conversi...Timer 3 Trigger Out event

Rank1

ChannelChannel 10

Sampling Time239.5 Cycles

Rank2

ChannelChannel 11

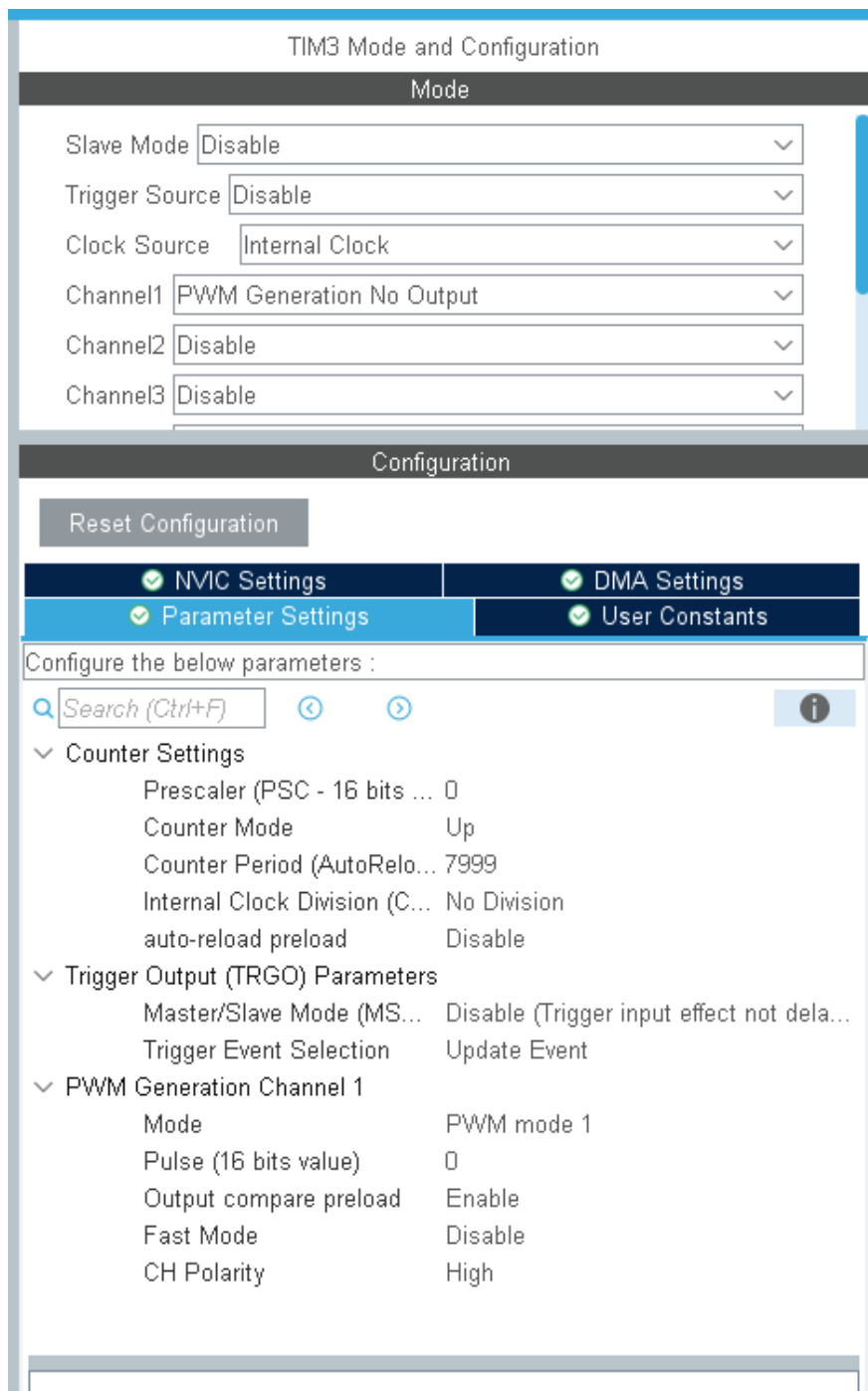
Sampling Time239.5 Cycles

ADC_Injected_ConversionMode

Enable Injected Conversi...Disable

WatchDog

Enable Analog WatchDo...☐



Tworzymy tablice do której DMA będzie zapisywac odczyt z ADC:

```
uint16_t joystick[2]
```

W głównej funkcji programu main() łączymy naszą zmienną z DMA i konkretnym przetwornikiem

```
HAL_ADC_Start_DMA(&hadc1, (uint32_t*)joystick, 2);
```

Deklarujemy strukture point która będzie zawierac współrzędne

```
typedef struct {
uint8_t x;
uint8_t y;
} Point;
```

Funkcja handle_joystic_input aktualizuje współrzędne kierunku w strukturze Point na podstawie wejść z joysticka


```

void handle_joystick_input(void)
{

if (joystick[0] < 1500 && direction.y == 0)
{
// "dół"
direction.x = 0;
direction.y = 1;
}
else if (joystick[0] > 2500 && direction.y == 0)
{
// "góra"
direction.x = 0;
direction.y = -1;
}

else if (joystick[1] < 1500 && direction.x == 0)
{
direction.x = -1;
direction.y = 0;
}
else if (joystick[1] > 2500 && direction.x == 0)
{
direction.x = 1;
direction.y = 0;
}
}

```

Funkcja move_snake korzysta z wartości direction.x i direction.y aby obliczyć nową pozycję głowy węża

```

void move_snake(void)
{
Point new_head = { snake[0].x + direction.x, snake[0].y + direction.y };
if (check_collision(new_head))
{
game_over = true;
return;
}
if (new_head.x == food.x && new_head.y == food.y)
{
snake_length++;
generate_food();
}
for (int i = snake_length - 1; i > 0; i--)
{
snake[i] = snake[i - 1];
}
snake[0] = new_head;
}

```

Na podstawie new_head sprawdzane są kolizje, wąż porusza się po planszy oraz aktualizowany jest wyświetlacz

Po przemieszczeniu węża w funkcji move_snake, zmieniona tablica snake jest używana do aktualizacji wyświetlacza

```
void update_display(void)
{
    LCD_clrScr();
    LCD_setPixel(food.x, food.y, true);
    for (uint8_t i = 0; i < snake_length; i++)
    {
        LCD_setPixel(snake[i].x, snake[i].y, true);
    }
    LCD_refreshScr();
}
```

Cały proces jest wykonywany w pętli głównej while w main()

```
while (1)
{
    if (snakeTimer == 0)
    {
        if (gameTimerState == 1 && !game_over)
        {
            handle_joystick_input();
            move_snake();
            update_display();
        }
        else if (game_over)
        {
            LCD_clrScr();

            LCD_refreshScr();

            game_over = false;
            snake_length = 3;
            direction.x = 1;
            direction.y = 0;
            snake[0].x = 40; snake[0].y = 24;
            snake[1].x = 39; snake[1].y = 24;
            snake[2].x = 38; snake[2].y = 24;
            generate_food();
        }
    }
}
```

```
snakeTimer = 10;  
}
```

W petli main obsługujemy dwa główne stany dla gry, czyli:

GameTimerState – decyduje czy gra ma być uruchomiona czy zapauzowana, sterujemy tą flagą za pomocą komend wysyłanych w protokole komunikacyjnym.

game_over – informuje, czy gra została zakończona i trzeba na nowo ją obsłużyć.